JOUR 2

Apprentissage par renforcement, estimation statistique apprise et atelier pratique

28 juillet 2026

Intervenants et thèmes principaux :
Introduction au Reinforcement Learning
Kyunghyun Cho — Statistics Upside Down
Miruna Pislar — Atelier pratique RL

Avant-propos

Ce document restitue, dans un format pédagogique et objectif, la deuxième journée de l'Eastern European Machine Learning Summer School (EEML) 2026, suivie à distance. Comme pour la journée précédente, il ne s'agit pas d'une transcription du direct, mais d'une réorganisation thématique du contenu, reformulée dans un registre académique neutre.

La journée a comporté trois temps distincts. Le premier a introduit l'apprentissage par renforcement (Reinforcement Learning, RL), depuis ses fondements formels jusqu'à ses usages les plus récents dans l'entraînement des grands modèles de langage. Le second, présenté par le chercheur Kyunghyun Cho sous le titre « Statistics Upside Down », a proposé une réflexion sur l'apprentissage automatique d'estimateurs statistiques, en remplacement de leur dérivation analytique traditionnelle. Le troisième temps était un atelier pratique animé par Miruna Pislar (Google DeepMind) et son collègue Seba, consacré à l'implémentation progressive d'un agent RL, de l'algorithme REINFORCE jusqu'aux bases de PPO et GRPO.

Comme pour le document de la journée 1, le corps du texte présente une synthèse concise de chaque chapitre, tandis que les développements plus détaillés sont regroupés en annexe, avec renvoi explicite depuis chaque chapitre.

Partie I — Introduction à l'apprentissage par renforcement

Cette première intervention de la journée a été assurée par un chercheur d'Isomorphic Labs, spécialisé dans la découverte de molécules thérapeutiques, et désigné plus tard dans la journée sous le prénom d'Andreas par une autre intervenante. Ce prénom est repris ci-après par commodité, faute d'une identification plus complète dans les notes disponibles.

Chapitre 1. Qu'est-ce que le RL ? Cadre général et applications

L'apprentissage par renforcement (Reinforcement Learning, RL) est présenté comme un changement de perspective par rapport à l'apprentissage supervisé : au lieu d'apprendre une fonction entre des entrées et des sorties connues, un agent apprend un comportement en interagissant avec un environnement, selon une boucle état → action → récompense.

Applications évoquées en introduction

- Les systèmes de recommandation par bandits (Spotify, Amazon, recommandation d'actualités), présentés comme l'application du RL la plus discrète mais la plus rentable.
- AlphaZero, capable d'atteindre un niveau de jeu supérieur dans plusieurs jeux de plateau, notamment le Go, où certains de ses coups ont surpris les experts humains.
- Le contrôle de champs de fusion nucléaire superchauffés, où le RL est utilisé pour stabiliser le plasma.
- L'apprentissage par renforcement à partir de retours humains (RLHF) et ses variantes plus récentes, aujourd'hui présents dans la plupart des grands modèles de langage utilisés chaque semaine par plus d'un milliard de personnes.

Détails et comparaison terme à terme entre apprentissage supervisé et RL : voir Annexe A.1.

Chapitre 2. Formaliser le problème : le processus de décision markovien

Le problème du RL est formalisé par un processus de décision markovien (Markov Decision Process, MDP), défini par cinq éléments : un ensemble d'états S, un ensemble d'actions A, une fonction de transition, une fonction de récompense, et un facteur d'actualisation γ (gamma).

Idées clés

- La propriété markovienne suppose que le futur ne dépend que de l'état présent, et non de l'historique complet (système « sans mémoire »). En pratique, les agents contournent souvent cette contrainte en empilant plusieurs états successifs pour simuler une forme de mémoire.
- L'agent cherche une politique — un comportement — qui maximise le retour, c'est-à-dire la somme des récompenses obtenues le long d'une trajectoire complète, et non la seule récompense immédiate.
- Le facteur d'actualisation γ détermine l'importance relative des récompenses futures par rapport aux récompenses immédiates.

Détails et exemple du monde en grille (banane, pièce, états terminaux) illustrant le rôle du facteur d'actualisation : voir Annexe A.2.

Chapitre 3. Bandits multi-bras et dilemme exploration/exploitation

Les bandits multi-bras (multi-armed bandits) constituent la version la plus simple du RL : un agent doit répartir ses essais entre plusieurs options (par analogie, plusieurs machines à sous) afin d'identifier celle qui rapporte le plus, sans notion d'état ni de transition.

Idées clés

- L'exploration consiste à essayer de nouvelles actions pour recueillir de l'information sur leur valeur.
- L'exploitation consiste à utiliser les actions déjà connues pour être les plus profitables.
- L'équilibre entre exploration et exploitation constitue l'un des problèmes fondamentaux de tout algorithme de RL : trop d'exploitation prématurée risque de figer l'agent sur une solution sous-optimale ; trop d'exploration retarde l'obtention de bons résultats.

Chapitre 4. Apprendre une politique : Policy Gradient et fonctions de valeur

Une première famille d'approches consiste à paramétrer directement la politique — typiquement par un réseau de neurones — et à optimiser ses paramètres par descente de gradient, de manière à maximiser la récompense attendue. C'est le principe du Policy Gradient.

Limite et notions complémentaires

- Dans sa forme la plus simple, le Policy Gradient attribue la même récompense à l'ensemble d'une trajectoire : le modèle ne sait donc pas distinguer précisément quelles actions ont réellement contribué au succès ou à l'échec (problème d'attribution du crédit).
- La fonction de valeur (Value Function) estime la qualité d'un état donné.
- La Q-valeur (Q-value) estime la qualité d'un couple état-action, ce qui permet ensuite de choisir l'action optimale dans chaque état.

Détails sur le calcul du retour et le rôle du gradient dans l'apprentissage de la politique : voir Annexe A.3.

Chapitre 5. Estimer les valeurs : programmation dynamique, Monte Carlo, différence temporelle

Trois grandes méthodes permettent d'estimer les fonctions de valeur d'un environnement.

Méthode	Principe	Caractéristiques
Programmation dynamique	Suppose une connaissance parfaite de l'environnement (transitions et récompenses)	Très efficace, mais rarement applicable en pratique
Monte Carlo	Attend la fin de l'épisode et utilise le retour réellement observé	Faible biais, variance élevée
Différence temporelle (TD)	Met à jour l'estimation immédiatement, sans attendre la fin de l'épisode	Variance plus faible, biais plus élevé ; fondement de nombreux algorithmes modernes

Chapitre 6. Familles d'algorithmes modernes : DQN, Actor-Critic, PPO

Model-free vs. model-based

- Les approches model-free (sans modèle) apprennent directement à agir, sans modéliser le fonctionnement de l'environnement. Exemples cités : PPO, DQN, SAC.
- Les approches model-based (avec modèle) apprennent en plus un modèle de l'environnement, capable de prédire le prochain état et la récompense associée, ce qui permet la planification et la simulation. Exemples cités : Dreamer, MuZero. Avantage : meilleure efficacité d'apprentissage. Inconvénient : si le modèle appris est erroné, les prédictions et décisions qui en découlent se dégradent.

On-policy vs. off-policy

- Les méthodes on-policy apprennent uniquement à partir des actions réellement exécutées par la politique courante (exemple : PPO).
- Les méthodes off-policy peuvent apprendre à partir d'expériences passées, même générées par une politique différente (exemples : Q-learning, DQN).

Trois architectures clés

- DQN (Deep Q-Network) : combine un réseau estimant les Q-valeurs, un mécanisme de rejeu d'expérience (Experience Replay) et un réseau cible (Target Network) ; ces ingrédients ont permis aux agents de DeepMind de dominer un large ensemble de jeux Atari.
- Actor-Critic : combine deux réseaux — l'Actor, qui choisit les actions, et le Critic, qui évalue leur qualité — ce dernier accélérant significativement l'apprentissage de l'Actor.
- Un bonus d'entropie est souvent ajouté à l'objectif d'entraînement pour encourager l'exploration : une politique très déterministe a une faible entropie, une politique très aléatoire une entropie élevée ; l'entropie est généralement élevée en début d'apprentissage puis réduite progressivement.

PPO (Proximal Policy Optimization) est présenté comme l'un des algorithmes les plus utilisés aujourd'hui : son principe est d'empêcher la politique de changer trop brutalement d'une mise à jour à l'autre, en limitant (« clipping ») l'amplitude des mises à jour, ce qui stabilise l'entraînement. C'est notamment cet algorithme qui a popularisé l'usage du RL pour l'entraînement des grands modèles de langage.

Chapitre 7. Le RL au service des grands modèles de langage

La dernière partie de cette intervention relie les concepts précédents à l'entraînement des grands modèles de langage (LLM).

Idées clés

- L'approche historique est le RLHF (Reinforcement Learning from Human Feedback) : des évaluateurs humains notent les réponses du modèle, et ces notes servent de signal de récompense.
- Une approche plus récente repose sur des récompenses vérifiables : tests automatiques de code, vérification de résultats mathématiques, réponses objectivement correctes — moins coûteuses, plus facilement mises à l'échelle et moins sensibles aux biais humains que le RLHF classique.
- DeepSeek R1 est cité comme illustration récente de cette approche : entraînement fondé sur des récompenses objectives, supervision humaine finale réduite, et émergence spontanée de comportements de raisonnement et d'auto-correction (décrits comme des « aha moments »).
- GRPO (Group Relative Policy Optimization) est présenté comme une évolution qui se passe du réseau Critic : plutôt que d'entraîner un réseau de valeur séparé, plusieurs réponses sont générées pour un même prompt, comparées entre elles, et leurs scores normalisés servent de signal d'apprentissage — ce qui réduit la mémoire nécessaire et accélère l'entraînement.
- L'intervenant a également évoqué la génération automatique d'environnements interactifs à partir d'un simple prompt textuel (par exemple un monde 3D), ouvrant de nouvelles perspectives pour l'entraînement d'agents RL.

Partie II — Statistics Upside Down : apprendre les estimateurs statistiques

Cette seconde intervention, assurée par le chercheur Kyunghyun Cho, change radicalement de sujet : elle porte sur une méthode alternative de conception d'estimateurs statistiques, fondée sur l'apprentissage automatique plutôt que sur la dérivation analytique.

Chapitre 8. Le biais caché de l'écart-type échantillonné

L'intervenant part d'une observation volontairement provocatrice : l'écart-type calculé sur un échantillon n'est pas un estimateur non biaisé de l'écart-type de la population, même après la correction de Bessel appliquée à la variance.

Idées clés

- La variance d'échantillon peut être corrigée pour devenir un estimateur non biaisé de la variance de la population (correction de Bessel, division par $n - 1$ plutôt que par n).
- L'écart-type, étant la racine carrée de la variance, reste néanmoins biaisé, car la racine carrée est une fonction non linéaire : en général, l'espérance d'une racine carrée n'est pas égale à la racine carrée de l'espérance.
- En conséquence, l'écart-type échantillonné a tendance à sous-estimer systématiquement l'écart-type réel de la population.
- Ce constat motive une question de recherche : est-il possible de construire un meilleur estimateur, qui minimise à la fois le biais et la variance ?

Détails et rappel du compromis biais-variance (décomposition du MSE) : voir Annexe B.1.

Chapitre 9. Apprendre un estimateur : le Set Transformer++

Plutôt que de chercher analytiquement une meilleure formule, l'intervenant et son collaborateur Maxim proposent de reformuler le problème comme une tâche d'apprentissage supervisé standard : entraîner un réseau de neurones à prédire directement l'écart-type réel d'une population à partir d'un échantillon.

Méthode et résultats

- Génération des données d'entraînement : on choisit aléatoirement une distribution (par exemple un mélange de deux lois normales) dont l'écart-type réel est connu exactement, on en tire un échantillon, puis on entraîne le réseau à minimiser l'erreur quadratique moyenne (MSE) entre sa prédiction et la valeur réelle.
- Choix architectural : les données d'entrée forment un ensemble (set) et non une séquence ordonnée ; le modèle doit donc être invariant par permutation. Les auteurs utilisent un Set Transformer++, une architecture conçue pour ce type de données.
- Évaluation : le modèle, entraîné sur des mélanges de gaussiennes, est testé sur des distributions jamais vues en entraînement (log-normales, à queues épaisses).
- Résultat : le Set Transformer++ obtient un estimateur presque non biaisé et de variance plus faible que les estimateurs classiques (écart-type avec ou sans correction de Bessel), en particulier sur des distributions proches de celles vues à l'entraînement ; les performances se dégradent, comme attendu, à mesure que l'on s'éloigne de cette distribution d'entraînement.

Chapitre 10. Généraliser l'idée : estimer l'information mutuelle avec MIST

La même philosophie est ensuite appliquée à un problème nettement plus difficile : l'estimation de l'information mutuelle entre deux variables aléatoires, une mesure de la quantité d'information partagée entre elles (par exemple entre une image et sa légende), réputée difficile à estimer lorsque les distributions sont complexes.

Méthode et résultats

- Génération des données d'entraînement : à partir d'une procédure proposée par Grabowski et al. (2023), on part de distributions bivariées normales dont l'information mutuelle exacte est calculable, puis on leur applique des transformations non linéaires mais inversibles (normalizing flows), ce qui permet de générer des échantillons complexes tout en connaissant exactement leur information mutuelle réelle.
- Un unique réseau, fondé sur des mécanismes d'attention, est entraîné sur un grand nombre de ces problèmes synthétiques (tailles d'échantillon de 10 à 500, dimensions de 2 à 32, information mutuelle variant de 0 à 40 nats), pour produire un modèle baptisé MIST (Mutual Information by Supervised Training).
- Comparé aux estimateurs classiques (CCA, KSG, InfoNCE, InfoNet), MIST obtient un biais et une variance plus faibles, avec un avantage particulièrement marqué lorsque la dimensionnalité augmente : contrairement aux méthodes fondées sur l'estimation de densité (comme KSG), dont les besoins en échantillons croissent rapidement avec la dimension (malédiction de la dimensionnalité), les besoins de MIST restent globalement stables.
- Le modèle est également entraîné par régression quantile (quantile regression, via une fonction de perte dite pinball loss), ce qui lui permet de produire, en plus d'une estimation ponctuelle, un intervalle d'incertitude bien calibré — un avantage obtenu à faible coût supplémentaire par rapport à des méthodes de calibration a posteriori telles que le bootstrap.

Détails sur la calibration de l'incertitude et les limites de généralisation hors du domaine d'entraînement : voir Annexe B.2.

Chapitre 11. Portée, limites et message philosophique

L'intervenant résume sa démarche par une formule : « Statistics Upside Down » — au lieu de partir d'une question (« quelle est la bonne formule ? »), il propose de partir d'un problème d'apprentissage automatique (« comment apprendre automatiquement la meilleure formule ? »).

Idées clés

- Cette approche ne remet pas en cause les fondements de la statistique classique ; elle propose une alternative pratique lorsque la puissance de calcul et les données simulées sont disponibles en abondance.
- Elle nécessite de définir un prior (une distribution sur l'espace des problèmes possibles) : les auteurs assument ce choix en le comparant aux hypothèses tout aussi fortes, mais souvent implicites et non vérifiables, sur lesquelles reposent les estimateurs analytiques classiques.
- Les estimateurs ainsi appris ne sont fiables qu'à proximité de leur distribution d'entraînement ; leur qualité se dégrade lorsqu'on s'en éloigne fortement.
- Cette démarche brouille la frontière entre statistique classique et apprentissage profond, en traitant la conception d'un estimateur comme un problème d'optimisation résolu par descente de gradient plutôt que par démonstration mathématique.

Partie III — Atelier pratique : construire un agent RL

Le troisième temps de la journée était un atelier pratique animé par Miruna Pislar, ingénieure de recherche chez Google DeepMind, accompagnée de son collègue Seba et d'une équipe d'assistants (TAs).

Chapitre 12. Objectifs du tutoriel et environnement CliffWalking

Objectifs annoncés

- Implémenter l'algorithme REINFORCE (Policy Gradient) à partir de zéro.
- Comprendre par la pratique les principes de PPO puis de GRPO.
- Appliquer ces algorithmes au fine-tuning d'un petit modèle de langage (Qwen, 500 millions de paramètres).

L'animatrice insiste sur l'idée que la compréhension du RL passe par sa mise en pratique, et encourage les participants à solliciter les assistants présents plutôt qu'à dépendre exclusivement des assistants LLM.

L'environnement CliffWalking

- Une grille de 4 lignes sur 12 colonnes, où l'agent (représenté par un elfe) doit se déplacer d'un point de départ (une chaise) vers un objectif (un cookie), en évitant une falaise représentée par une rangée de cases.
- Quatre actions possibles, codées par des entiers : 0 = haut, 1 = droite, 2 = bas, 3 = gauche.
- Un état correspond à la position de l'agent dans la grille (ligne, colonne), convertie en un entier unique.
- Récompenses : −1 pour un déplacement normal, −100 en cas de chute dans la falaise (l'épisode se termine alors), 0 lorsque l'agent atteint l'objectif.
- La politique constitue le « cerveau » de l'agent : une fonction qui associe à chaque état une distribution de probabilité sur les actions possibles ; au début de l'entraînement, cette distribution est presque uniforme (comportement quasi aléatoire), puis elle se concentre progressivement sur les meilleures actions.
- Le retour total maximal atteignable dans cet environnement est de −13 (un déplacement vers le haut, puis douze déplacements vers la droite, le dernier pas rapportant une récompense nulle) : un score négatif mais optimal, puisqu'il minimise la pénalité cumulée.

Chapitre 13. De REINFORCE vers PPO, GRPO et le fine-tuning d'un LLM

Le tutoriel est structuré en deux parties progressives.

Partie 1 — Construire un agent RL « from scratch »

- Représentation des états de l'environnement CliffWalking.
- Définition d'une politique sous la forme d'un petit réseau de neurones.
- Interaction avec l'environnement : à chaque étape, l'état courant est fourni à la politique, qui choisit une action ; l'environnement renvoie alors un nouvel état et une récompense.
- Collecte de trajectoires complètes et calcul du retour associé.
- Mise à jour des paramètres de la politique par Policy Gradient (algorithme REINFORCE).

Partie 2 — Vers le fine-tuning d'un modèle de langage

- Cette seconde partie s'appuie sur les bases posées en partie 1 pour introduire PPO, GRPO, le RLHF et, plus largement, le post-entraînement des grands modèles de langage.
- Contrairement à l'environnement CliffWalking, l'entraînement d'un LLM ne dispose pas d'un environnement fournissant directement une récompense ; celle-ci doit être définie ou apprise séparément (juge humain, modèle de récompense, ou critère vérifiable selon les approches vues en partie I).

Cette progression pédagogique — d'un environnement de grille simple jusqu'au fine-tuning d'un modèle de langage — relie directement les notions théoriques présentées le matin (Policy Gradient, PPO, GRPO, RLHF) à une implémentation concrète, dans la continuité des chapitres 4 à 7.

Conclusion générale

Bien que couvrant des sujets en apparence distincts, les trois temps de cette deuxième journée partagent un même fil conducteur : la question de savoir dans quelle mesure certaines démarches, traditionnellement conçues « à la main » par des experts, peuvent être remplacées ou complétées par un processus d'apprentissage automatique.

Fils conducteurs de la journée

- En apprentissage par renforcement, l'agent découvre lui-même, par essais-erreurs, un comportement optimal, sans qu'on lui indique explicitement la bonne action à chaque étape.
- Dans l'exposé « Statistics Upside Down », des estimateurs statistiques eux-mêmes sont appris par un réseau de neurones plutôt que dérivés analytiquement, avec des gains mesurables de biais et de variance sur les cas étudiés.
- L'atelier pratique a permis de relier ces principes théoriques à une implémentation concrète, de l'algorithme REINFORCE jusqu'aux briques nécessaires au fine-tuning d'un grand modèle de langage par renforcement.
- Le fil qui relie ces trois interventions rejoint également, de façon plus large, le thème de la première journée : dans les deux cas, il s'agit de remplacer une construction manuelle (caractéristiques, formules statistiques) par un objectif d'optimisation appris directement à partir des données.

Ce document restitue les notions abordées dans un cadre pédagogique et objectif ; certains échanges avec le public, notamment en fin de tutoriel, n'ont pu être que partiellement restitués dans les notes disponibles.

Annexe — Approfondissements par chapitre

Cette annexe reprend, chapitre par chapitre, les développements, exemples et éléments chiffrés qui étayent les synthèses présentées dans le corps du document.

A.1 — Apprentissage supervisé et RL (complément du chapitre 1)

Aspect	Apprentissage supervisé	Apprentissage par renforcement
Nature des données	Paires (X, Y) fournies à l'avance	Générées par l'agent lui-même, au fil de ses interactions avec l'environnement
Signal d'apprentissage	Étiquette correcte connue (Y)	Récompense, souvent conçue par un humain, sans notion de « réponse correcte » unique
Nature du signal	Immédiat (associé à chaque exemple)	Parfois différé de plusieurs étapes
Objectif	Apprendre une fonction de X vers Y	Apprendre un comportement (politique) qui maximise le retour à long terme

A.2 — Le processus de décision markovien : exemple illustratif (complément du chapitre 2)

L'exemple du monde en grille présenté en séance comporte un état gagnant valant +100, deux états terminaux de pénalité (−100 chacun), une case « banane » valant −1, et une case « pièce » valant +1. Plusieurs chemins permettent d'atteindre l'état gagnant : un chemin passant par la banane obtient un retour total de 99 (100 − 1) si l'on additionne simplement les récompenses ; un chemin passant par la pièce ou un chemin direct obtiennent chacun un retour de 100.

C'est à ce stade qu'intervient le facteur d'actualisation γ : selon sa valeur, l'algorithme accorde plus ou moins d'importance à des récompenses obtenues rapidement plutôt que tardivement. Un γ proche de 0 privilégie fortement les récompenses immédiates ; un γ proche de 1 accorde presque autant de poids aux récompenses lointaines qu'aux récompenses immédiates. Ce mécanisme permet notamment de départager deux trajectoires de même retour total, mais de longueurs différentes, en favorisant la plus courte.

A.3 — Policy Gradient : le calcul du retour (complément du chapitre 4)

Une politique donne, à chaque pas de temps, une décision locale (quelle action prendre dans l'état courant), mais l'objectif de l'agent porte sur l'ensemble de la trajectoire : c'est la somme actualisée des récompenses futures, le retour, que l'on cherche à maximiser plutôt qu'une récompense isolée. Le Policy Gradient ajuste directement les paramètres du réseau représentant la politique dans la direction qui augmente l'espérance de ce retour.

La limite mentionnée en séance — l'ensemble d'une trajectoire recevant la même récompense — signifie que, dans sa version la plus simple, l'algorithme ne distingue pas les actions réellement décisives des actions neutres ou légèrement négatives au sein d'une trajectoire globalement réussie. Cette difficulté, dite d'attribution du crédit (credit assignment), motive en partie l'introduction des fonctions de valeur et des architectures Actor-Critic présentées au chapitre 6.

B.1 — Le compromis biais-variance (complément du chapitre 8)

Un bon estimateur doit conjuguer un faible biais (l'espérance de l'estimateur doit être proche de la vraie valeur) et une faible variance (l'estimateur doit rester stable d'un échantillon à l'autre). Ces deux exigences sont résumées par la décomposition classique de l'erreur quadratique moyenne :

$MSE = Biais^2 + Variance$

Rechercher un bon estimateur revient donc à minimiser conjointement le carré du biais et la variance, c'est-à-dire à minimiser l'erreur quadratique moyenne. L'écart-type échantillonné présente un biais non négligeable (généralement une sous-estimation) mais une variance relativement faible ; la question posée par l'intervenant est de savoir si un estimateur appris peut améliorer ce compromis dans son ensemble, plutôt que d'échanger un défaut contre un autre.

B.2 — MIST : calibration et limites (complément du chapitre 10)

Au-delà de l'estimation ponctuelle de l'information mutuelle, le modèle MIST est entraîné par régression quantile simultanée (prédiction du 10e, du 50e, du 90e percentile, etc., à l'aide d'une fonction de perte dite pinball loss), ce qui lui permet de produire une estimation accompagnée d'un intervalle d'incertitude, par exemple « information mutuelle = 2,5, avec un intervalle de confiance associé », plutôt qu'une seule valeur ponctuelle.

Sur le plan de la calibration, les courbes présentées en séance montrent qu'un modèle entraîné uniquement à minimiser l'erreur quadratique moyenne, puis recalibré a posteriori par une méthode de rééchantillonnage (bootstrap), reste bien calibré sur les valeurs faibles mais se dégrade sur les valeurs plus élevées ; le modèle entraîné directement par régression quantile obtient, lui, une calibration presque parfaite, avec un coût de mise en œuvre supplémentaire minime.

Les limites reconnues par l'intervenant sont les suivantes : la performance du modèle dépend du prior choisi sur l'espace des problèmes (la famille de distributions utilisée pour générer les données d'entraînement) et se dégrade à mesure que les distributions rencontrées en test s'éloignent de ce prior ; de plus, le modèle a tendance à sous-estimer l'information mutuelle dans les cas extrêmes, en raison de la plage dynamique finie du réseau de neurones utilisé. L'intervenant relativise néanmoins cette limite en notant que les estimateurs classiques reposent eux aussi sur des hypothèses fortes concernant la distribution sous-jacente, rarement vérifiables en pratique — la question n'étant donc pas d'éviter toute hypothèse, mais de choisir consciemment celles que l'on formule.